

Пояснительная записка к проекту

по курсу: Структуры и алгоритмы обработки данных

по теме: Задачи маршрутизации транспорта (Vehicle Routing Problem)

организация: Филиал МГУ имени М.В. Ломоносова в городе Дубне

направление: Методы и технологии обработки данных в гетерогенных вычислительных средах

выполнила: студентка 5-го курса **Брусилова Ирина Владимировна**

дата: 4 июня 2026 г.

Введение

Задача маршрутизации транспорта (Vehicle Routing Problem, VRP) занимает центральное место в комбинаторной оптимизации и операционных исследованиях. В общей постановке требуется построить оптимальные маршруты для парка транспортных средств, обслуживающих множество географически рассредоточенных клиентов при соблюдении ряда ограничений. Задача была впервые формализована Данцигом и Рамзером в 1959 году [1] как обобщение знаменитой задачи коммивояжёра (TSP) на случай нескольких транспортных средств.

Практическую значимость VRP трудно переоценить. Задача возникает при организации доставки товаров, планировании маршрутов общественного транспорта, сборе мусора, обслуживании оборудования на выезде и в десятках других прикладных контекстов. По различным оценкам, транспортные расходы составляют от 10 до 20% конечной стоимости товаров [2], поэтому даже незначительное улучшение маршрутов ведёт к существенной экономии.

С вычислительной точки зрения VRP является NP-трудной задачей [3]: не известно полиномиального алгоритма, гарантирующего точное решение, а перебор всех возможных маршрутов неосуществим при практических размерностях. Это обстоятельство делает разработку эффективных эвристических и метаэвристических методов принципиально важным направлением исследований.

В настоящей работе рассматриваются два варианта постановки. *Capacitated VRP* (CVRP) — классическая задача с ограничением по грузоподъёмности транспортных средств: каждый клиент имеет известный спрос, и суммарный спрос по любому маршруту не должен превышать вместимость транспортного средства. Цель — минимизировать суммарную длину всех маршрутов. *VRP with Time Windows* (VRPTW) — расширение CVRP, в котором каждый клиент должен быть обслужен в заданном временном интервале $[a_i, b_i]$; нарушение окна недопустимо. Данный вариант существенно ужесточает ограничения и требует более изощрённых алгоритмов.

Цель работы — реализовать и исследовать спектр алгоритмов для решения задач CVRP и VRPTW: от простых конструктивных эвристик до метаэвристик и собственной гибридной модификации. Основные задачи: изучить классические подходы к VRP; реализовать на языке C++ набор алгоритмов различного уровня сложности; провести сравнительный эксперимент на стандартных бенчмарках; предложить и обосновать собственную модификацию.

Описание алгоритмов и структур данных

Архитектура проекта. Проект реализован на C++17 и структурирован в виде набора статических библиотек: `vrp_core`, `vrp_parsers`, `vrp_heuristics`, `vrp_local_search`, `vrp_metaheuristics`, `vrp_hybrid`. Сборка производится посредством CMake.

Основные структуры данных. Структура `Client` хранит идентификатор узла, евклидовы координаты, спрос, а также поля временного окна (`ready`, `due`, `service`) для задач VRPTW. Структура `Instance` объединяет список клиентов (индекс 0 зарезервирован под депо), вместимость транспортного средства и максимальное число машин. Класс `DistanceMatrix` реализует предвычисленную матрицу евклидовых расстояний в плоском представлении (row-major): это обеспечивает обращение за $O(1)$ при однократных затратах $O(n^2)$ на построение. Структура `Route` содержит упорядоченный список клиентов в маршруте (без депо на концах — оно подразумевается). Структура `Solution` хранит вектор маршрутов и предоставляет методы вычисления суммарной длины и проверки допустимости.

Поддерживаются два формата входных данных: TSPLIB (`.vrp`) — формат CVRPLIB для задач

CVRP, и Solomon (.txt) — формат для задач VRPTW из классического набора бенчмарков Соломона [5].

Конструктивные эвристики. *Nearest Neighbor (NN)* — жадный алгоритм, строящий маршруты последовательно: из текущей позиции (изначально — депо) выбирается ближайший непосещённый клиент, вмещающийся по нагрузке. Когда ни один клиент не помещается в текущий маршрут, открывается новый. Алгоритм прост и быстр, однако склонен формировать неэффективные маршруты, так как «жадный» выбор на каждом шаге не учитывает глобальную структуру задачи.

Clarke-Wright (CW) [4] — параллельная версия алгоритма экономий. На первом шаге формируется стартовое решение: каждый клиент обслуживается отдельным маршрутом вида «депо — клиент — депо». Для каждой пары клиентов (i, j) вычисляется экономия $s(i, j) = d_{0i} + d_{0j} - d_{ij}$, где d_{0k} — расстояние от депо до клиента k . Список экономий сортируется по убыванию; жадно применяются слияния маршрутов при условии, что i является хвостом своего маршрута, j — головой, и суммарный спрос не превышает вместимость. Алгоритм значительно превосходит NN по качеству решений.

Локальный поиск. *2-opt* — классический оператор улучшения маршрута. Для каждой пары рёбер $(a \rightarrow b, c \rightarrow d)$ одного маршрута проверяется, выгоден ли «разворот» сегмента $[b \dots c]$, то есть замена двух рёбер на $(a \rightarrow c, b \rightarrow d)$. Применяется стратегия «first improvement»: при первом обнаруженном улучшении сегмент разворачивается, поиск начинается заново. Процедура повторяется до локального оптимума.

Or-opt — обобщение 2-opt, переносящее связные сегменты из 1–3 клиентов в другую позицию внутри того же маршрута. Для каждого сегмента перебираются все возможные позиции вставки; применяется стратегия «best improvement» — выбирается наилучший ход за весь проход. Операторы применяются для $k = 1, 2, 3$ последовательно.

Relocate — межмаршрутный оператор: клиент переносится из одного маршрута в другой на оптимальную позицию при условии соблюдения вместимости. Перебираются все пары (клиент, маршрут-приёмник); применяется «best improvement».

Swap — межмаршрутный обмен: два клиента из разных маршрутов меняются местами (на тех же позициях), если оба маршрута остаются допустимы по вместимости.

Метаэвристики. *Simulated Annealing (SA)* [7] — метаэвристика, моделирующая процесс отжига. На каждой итерации к текущему решению применяется случайный оператор соседства (2-opt на случайном маршруте, or-opt или relocate); полученное решение оценивается взвешенной функцией $f = L + \lambda \cdot R$, где L — суммарная длина, R — число маршрутов, λ — штраф. Ухудшение принимается с вероятностью $e^{-\Delta f/T}$, где T — текущая температура. Реализовано адаптивное охлаждение: при обнаружении «плато» (отсутствие улучшений в течение заданного числа итераций) коэффициент охлаждения временно замедляется с α до $\alpha_{\text{boost}} > \alpha$, что позволяет дольше исследовать окрестность. Алгоритм останавливается при достижении минимальной температуры $T_{\min}$ или исчерпании лимита итераций без улучшения глобального оптимума.

Tabu Search (TS) [8] — метаэвристика, явно запрещающая недавно выполненные ходы. На каждой итерации выбирается лучший допустимый ход из всех возможных Relocate и Swap; ход запрещается на *tenure* итераций путём добавления записи в список запретов. Реализован *критерий аспирации*: запрещённый ход допускается, если он улучшает глобально лучшее найденное решение. После каждого хода к изменённым маршрутам применяется быстрый 2-opt.

Гибридный решатель (собственная модификация). Реализован конвейер: $CW \rightarrow 2\text{-opt} \rightarrow Or\text{-opt} \rightarrow Relocate+Swap \rightarrow SA$. На первом этапе Clarke-Wright формирует качественное начальное решение; затем применяется Variable Neighborhood Descent (VND): операторы локального поиска (2-opt, or-opt, relocate, swap) чередуются до тех пор, пока хотя бы один из них даёт улучшение. Финальный этап — SA с адаптивным охлаждением — позволяет выйти из локального оптимума. Ключевым элементом является взвешенная функция оценки $f = L + \lambda \cdot R$, используемая единообразно на этапах SA и TS: штраф λ за каждое дополнительное транспортное средство стимулирует алгоритм сокращать парк при незначительном увеличении пробега, что соответствует реальным экономическим условиям.

Теоретическое обоснование. Анализ вычислительной сложности

Сложность алгоритмов анализируется относительно n — числа клиентов, m — числа маршрутов, $L = n/m$ — средней длины маршрута.

DistanceMatrix. Построение: $O(n^2)$ по времени и памяти. Каждое обращение: $O(1)$.

Nearest Neighbor. На каждой итерации внешнего цикла (открытие нового маршрута) внутренний цикл выбора ближайшего клиента выполняется за $O(n)$; в худшем случае маршрут содержит $O(n)$ клиентов, то есть один маршрут строится за $O(n^2)$. Всего $O(m)$ маршрутов — итоговая сложность $O(n^2)$. Пространственная сложность: $O(n)$.

Clarke-Wright. Вычисление экономий: $O(n^2)$ пар, каждая — $O(1)$. Сортировка: $O(n^2 \log n)$. Жадное слияние: $O(n^2)$ в худшем случае. Итого: $O(n^2 \log n)$ по времени. Пространство: $O(n^2)$ для хранения экономий.

2-opt. Один проход по маршруту длины L : $O(L^2)$ пар рёбер. В худшем случае применяется $O(L^2)$ улучшений, что даёт $O(L^4)$ за один маршрут, однако на практике сходимость значительно быстрее. Для всего решения из m маршрутов: $O(m \cdot L^2)$ на проход. Пространство: $O(1)$ дополнительной памяти.

Or-opt. Один проход по маршруту длины L для фиксированного k : $O(L^2)$ (перебор сегментов и позиций вставки). Для $k \in \{1, 2, 3\}$: $O(L^2)$ в сумме. Для всего решения: $O(m \cdot L^2)$ на проход. Пространство: $O(L)$ для хранения извлечённого сегмента.

Relocate / Swap (InterRoute). На каждой итерации перебираются все пары (клиент в маршруте r_1 , маршрут-приёмник r_2): $O(n \cdot m \cdot L) = O(n^2)$ операций на итерацию. В худшем случае выполняется $O(n)$ итераций — итого $O(n^3)$. На практике число итераций невелико. Пространство: $O(1)$.

Simulated Annealing. Каждая итерация применяет один оператор соседства за $O(L)$ и пересчитывает стоимость за $O(m \cdot L) = O(n)$. При I итерациях итого $O(I \cdot n)$. Пространство: $O(n)$ для хранения копии решения.

Tabu Search. На каждой итерации вычисляются лучшие ходы Relocate ($O(n^2)$) и Swap ($O(n^2)$), после чего применяется 2-opt ($O(m \cdot L^2)$). При I итерациях: $O(I \cdot n^2)$. Пространство: $O(T_{\text{tenure}} \cdot n)$ для списка запретов.

Гибридный решатель. Доминирующий вклад вносит этап SA, обёрнутый вокруг VND. VND выполняет несколько полных проходов локального поиска — суммарно $O(n^3)$ в худшем случае за счёт InterRoute. Этап SA добавляет $O(I \cdot n)$. Итого: $O(n^3 + I \cdot n)$. Пространство: $O(n^2)$ (матрица расстояний).

Сводная таблица сложности приведена ниже (таблица 1).

Таблица 1: Вычислительная сложность алгоритмов

Алгоритм	Временная сложность	Пространственная
DistanceMatrix	$O(n^2)$	$O(n^2)$
Nearest Neighbor	$O(n^2)$	$O(n)$
Clarke-Wright	$O(n^2 \log n)$	$O(n^2)$
2-opt (один маршрут)	$O(L^4)$ worst, $O(L^2)$ avg	$O(1)$
Or-opt (один маршрут)	$O(L^2)$ на проход	$O(L)$
Relocate / Swap	$O(n^3)$ worst	$O(1)$
Simulated Annealing	$O(I \cdot n)$	$O(n)$
Tabu Search	$O(I \cdot n^2)$	$O(T_{\text{ten}} \cdot n)$
Гибрид (Hybrid)	$O(n^3 + I \cdot n)$	$O(n^2)$

Выбор взвешенной функции оценки $f = L + \lambda \cdot R$ обоснован следующим образом. В реальных задачах стоимость эксплуатации транспортного средства (амортизация, водитель) зачастую превышает переменные расходы на топливо. Включение штрафного слагаемого $\lambda \cdot R$ превращает задачу минимизации только расстояния в двухкритериальную: алгоритм стремится одновременно сокращать пробег и число машин. Параметр λ позволяет регулировать баланс между этими целями в зависимости от конкретного приложения.

Результаты работы алгоритмов

Эксперименты проводились на **83 стандартных бенчмарках**: 27 CVRP-инстансов из библиотеки CVRPLIB [6] (серия A) и 56 VRPTW-инстансов из наборов Соломона [5] (серии C, R, RC). Функция оценки: $f = L + 10 \cdot R$, где L — суммарная длина маршрутов, R — число маршрутов, штраф $\lambda = 10$.

Результаты на стандартных бенчмарках представлены в таблице 2.

Таблица 2: Сравнение алгоритмов на инстансах A-n32-k5 (CVRP) и C101 (VRPTW)

Алгоритм	A-n32-k5 ($n = 31$, cap=100)			C101 ($n = 100$, cap=200)		
	Маршр.	Длина	Время, мс	Маршр.	Длина	Время, мс
NN	5	1146.4	0.01	10	1311.5	0.02
CW	5	843.7	0.04	10	838.9	0.23
CW+2opt	5	830.7	0.02	10	826.1	0.24
SA	5	830.7	0.54	10	826.1	0.87
TS	5	828.7	0.86	10	822.0	7.02
Hybrid	5	828.7	0.48	10	821.1	1.04

Сводная статистика по всем 83 бенчмаркам представлена в таблице 3.

Таблица 3: Средние показатели алгоритмов по 83 инстансам (CVRP + VRPTW)

Алгоритм	Средняя длина	Среднее время, мс	Лучший на, инст.
NN	1190.2	0.016	—
CW	893.7	0.192	—
CW+2opt	880.6	0.196	3
SA	879.9	1.121	—
TS	874.7	3.883	17
Hybrid	874.0	1.214	63

Ключевые наблюдения.

Преимущество гибридного решателя. На 63 из 83 инстансов (75.9%) гибридный алгоритм показывает наилучшую длину маршрутов среди всех шести методов. С учётом инстансов, где гибридный делит первое место с другим алгоритмом, доля успешных исходов достигает **83%**. Ближайший конкурент — Tabu Search — лидирует лишь на 17 инстансах.

Качество vs. время. Гибридный решатель в среднем в **3.2 раза быстрее** Tabu Search (1.21 мс против 3.88 мс), при этом обеспечивая сопоставимое или лучшее качество. По качеству Hybrid превосходит SA, несмотря на близкое среднее время работы.

Различие по типу задачи. На CVRP-инстансах средняя длина Hybrid составляет 1080.8 ед., на VRPTW — 774.2 ед.; в обоих классах гибридный опережает все прочие методы. Tabu Search показывает чуть лучший результат на ряде CVRP-инстансов (средняя длина 1077.3), уступая однако в быстродействии.

Улучшение относительно NN. Гибридный решатель сокращает суммарную длину маршрутов на **25.9%** в среднем по сравнению с Nearest Neighbor (разброс: от 14.7% до 37.4% в зависимости от инстанса).

Заключение

В рамках работы была реализована и исследована система решателей задачи маршрутизации транспорта на языке C++17, охватывающая полный спектр алгоритмических подходов — от конструктивных эвристик до метаэвристик. Сравнительный эксперимент проводился на 83 стандартных бенчмарках двух классов: CVRP (серия A, CVRPLIB) и VRPTW (серии C, R, RC, Solomon).

Конструктивные эвристики (NN, CW) обеспечивают быстрое начальное решение. Clarke-Wright сокращает суммарную длину маршрутов относительно NN в среднем на 24%, а дополнительное применение 2-opt даёт ещё 1.7%. Метаэвристики — SA и TS — в ряде случаев превосходят локальный поиск, однако их выигрыш не превышает 1–2% при существенно большем времени работы: Tabu Search в среднем вдвое медленнее SA и почти в 20 раз медленнее CW+2opt.

Гибридный решатель (CW → VND → SA) демонстрирует наилучший итоговый баланс качества и скорости. На 83% бенчмарков (с учётом разделённых первых мест) гибрид выдаёт результат не хуже любого другого алгоритма из рассмотренных, при этом на 63 инстансах (75.9%) занимает безоговорочное первое место — против 17 у ближайшего конкурента, Tabu Search. Средняя длина маршрутов гибрида составляет 874.0 ед., что на 0.08% лучше, чем у TS (874.7 ед.); при этом гибрид в среднем быстрее TS в **3.2 раза** (1.21 мс против 3.88 мс), а на отдельных инстансах ускорение достигает 6.9×. Ключевым элементом модификации является взвешенная функция оценки $f = L + \lambda R$, которая стимулирует сокращение числа транспортных средств при умеренном увеличении пробега — это соответствует реальным экономическим условиям, где стоимость эксплуатации парка зачастую превышает переменные расходы на топливо.

Список литературы

1. Dantzig G.B., Ramser J.H. The Truck Dispatching Problem // Management Science. 1959. Vol. 6, No. 1. P. 80–91. DOI: [10.1287/mnsc.6.1.80](https://doi.org/10.1287/mnsc.6.1.80)
2. Toth P., Vigo D. (eds.) The Vehicle Routing Problem. SIAM, Philadelphia, 2002.
3. Lenstra J.K., Rinnooy Kan A.H.G. Complexity of vehicle routing and scheduling problems // Networks. 1981. Vol. 11, No. 2. P. 221–227.
4. Clarke G., Wright J.W. Scheduling of Vehicles from a Central Depot to a Number of Delivery Points // Operations Research. 1964. Vol. 12, No. 4. P. 568–581.
5. Solomon M.M. Algorithms for the Vehicle Routing and Scheduling Problems with Time Window Constraints // Operations Research. 1987. Vol. 35, No. 2. P. 254–265.
6. Christofides N., Mingozzi A., Toth P. The Vehicle Routing Problem // Combinatorial Optimization. Wiley, 1979. P. 315–338.
7. Kirkpatrick S., Gelatt C.D., Vecchi M.P. Optimization by Simulated Annealing // Science. 1983. Vol. 220, No. 4598. P. 671–680.
8. Glover F. Tabu Search – Part I // ORSA Journal on Computing. 1989. Vol. 1, No. 3. P. 190–206.
9. Laporte G. The Vehicle Routing Problem: An overview of exact and approximate algorithms // European Journal of Operational Research. 1992. Vol. 59, No. 3. P. 345–358.